AI Engineering and Claude Architect · Part I, Foundations

Generative AI and LLMs, Lesson 7

Generative AI Beyond Text, and Choosing a Model

Your classification route runs on the most capable model, at a cost of $18,000 per million requests. The cheapest model might do it for $450. Should you switch? The honest answer starts with a test, and not with a price list.

Module	AI02 of 13
Lesson	7 of 7, including the module revision
Prerequisites	AI02 L1–L6
Exam relevance	Vision and model comparisons are out of scope
Reading time	~35 minutes

Contents

1. Generative AI Beyond Text, and Choosing a Model
2. Module Revision

Lesson 7 of 7 · Generative AI and LLM Fundamentals

Generative AI Beyond Text, and Choosing a Model

Key idea

Generative models learn the distribution of their data well enough to sample new examples from it. GANs, VAEs and diffusion do that for images and more, while LLMs do it for token sequences. Multimodal models bring these together, by turning everything into vectors. For an engineer, the practical decision is which model to use, and **the answer comes from measuring cost per completed task on your own workload, not from a benchmark or a price list.**

> *Before: autoencoders (AI01 L4), the LLM (AI02 L3–L6). Teaches: GANs, VAEs, diffusion, multimodal models, Claude's model tiers, a method for choosing a model, and model routing. Used by: AI03 L4 (effort), AI10 L2 (gateways and routing), AI10 L5 (cost engineering), AI11 L4.*

The hook: is the expensive model worth it?

You classify support tickets. Each request has about 2,000 input tokens and 500 output tokens. Here's the cost per million requests, computed from Anthropic's listed prices in the October 2026 reference used by this course.

Model	Input per 1M tokens	Output per 1M tokens	Cost per 1M requests
Claude Fable 5.1	$10.00	$50.00	$45,000
Claude Opus 5.5	$4.00	$20.00	$18,000
Claude Sonnet 5.5	$2.00	$10.00	$9,000
Claude Haiku 5.5	$0.10	$0.50	$450

That's a factor of 100 between the top and the bottom. The tempting conclusion is to always use the cheapest one.

> *Key idea*
>
> **Price per token isn't cost per solved task.** If the cheaper model gets 15% of tickets wrong and a human then has to fix each one, the cheap route may be the expensive one. So you need a measurement before a decision. We'll get there, after a quick tour of what else generative AI covers.

> *Version-sensitive*
>
> Prices and models change. These come from a reference dated 6 October 2026, and Haiku 5.5's price applies to prompts up to 100K tokens. Check the current pricing page.

Intuition: a forger, a compressor and a restorer

Family	The analogy
GAN	A forger and a detective improve by competing
VAE	Compress an image to a small code, then redraw it from a code you sample
Diffusion	A restorer learns to remove noise, and then "restores" pure noise into a new image
LLM	An autocomplete, drawing one token at a time

Each one solves the same problem: **draw a new sample that looks as if it came from the training data.**

The generative families

GANs train two networks against each other. A generator makes samples, while a discriminator tries to tell them from real data.

VAEs extend autoencoders (AI01 L4). The encoder outputs a distribution over codes rather than a single point, so you can sample a new code and decode it.

Diffusion models learn to reverse a gradual noising process.

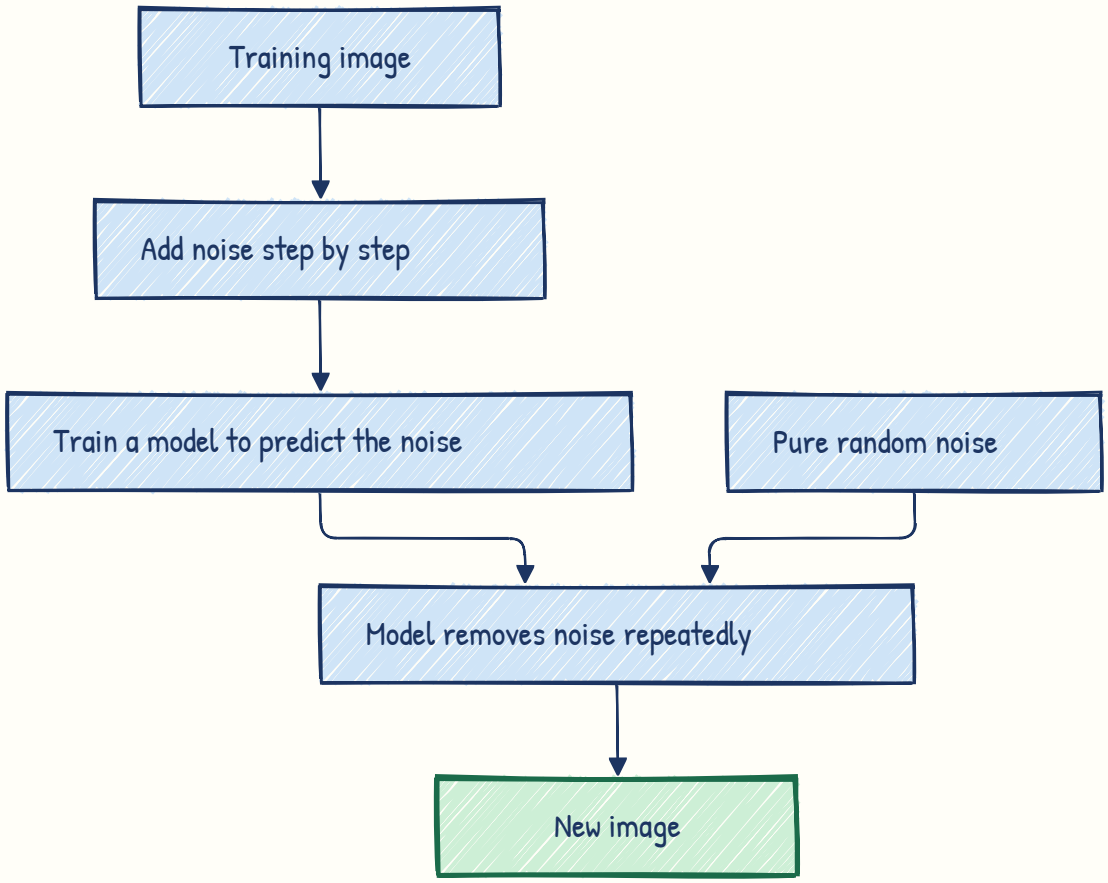


Figure 1. Diffusion. Training learns to undo small steps of added noise. Generation starts from pure noise and removes it step by step.

Family	Strength	Weakness	Typical use
GAN	Fast sampling	Training can be unstable	Image synthesis
VAE	Stable, with a smooth latent space	Often blurrier	Compression, representation learning
Diffusion	High sample quality	Many sampling steps	Image generation
Transformer LLM	Language and code	Plausibility without truth (AI02 L6)	Text, code, agents

Multimodal models

How can one model read text and images? **By turning everything into vectors.** An image can be divided into patches, with each patch embedded, so attention can then process image and text tokens together. Anthropic's Golden Gate Bridge feature fired on both the words and images of the bridge, which fits this picture. Claude accepts images and PDFs as input on the API, and those inputs consume tokens as well.

Claude's model tiers

The table above lists current generally available tiers. All have a 1M-token context window, according to the reference. Older models and other details are on the models page.

Tier	Described by Anthropic as	Good starting point for
Claude Fable 5.1	Its most capable widely released model	The most demanding reasoning and long-horizon agents
Claude Opus 5.5	The current Opus	Hard coding and agentic work
Claude Sonnet 5.5	Speed and capability for everyday coding, agents and enterprise work	High-volume, capable tasks
Claude Haiku 5.5	The current Haiku	Simple, high-volume tasks

The right-hand column is a starting point, and it's my recommendation rather than Anthropic's guarantee.

> ***Model differences break code***
>
> Models in this family aren't interchangeable behind one setting. For example, forced `tool_choice` returns a 400 on Opus 5.5, Sonnet 5.5 and Fable 5.1, and thinking can't be disabled on Opus 5.5. Sonnet 5.5 and Haiku 5.5 also have different effort defaults. **Test each model before routing to it, and don't simply swap IDs.** AI03 L4 covers these.

How to choose a model: a method

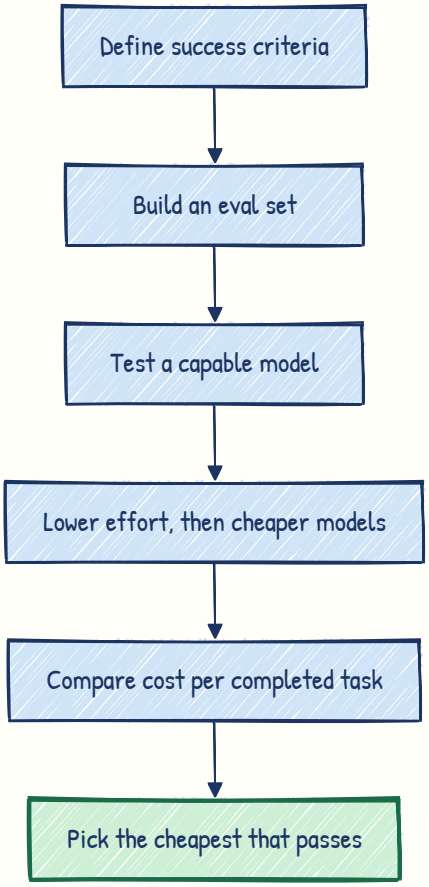


Figure 2. Choosing a model. Start from the task and an eval set, begin with a capable model, then step down only as far as your quality bar allows.

Why begin with the capable model? It tells you what good looks like. If it fails, the task or prompt has a problem that a cheaper model won't fix.

Anthropic's own API guidance adds a practical step: before building a multi-model cascade, **measure the most capable model at lower effort.** One model also means one cache namespace, since caches are model-scoped, so a cascade gives up cache reuse across models.

Cost per completed task

Here's the hook again, with the missing number. Suppose a human fix costs $2 per wrong ticket. The error rates below are **invented for illustration.**

Model	Model cost per ticket	Error rate	Human fix cost per ticket	Total per ticket
Opus 5.5	$0.0180	1%	$0.0200	**$0.0380**
Sonnet 5.5	$0.0090	2%	$0.0400	**$0.0490**
Haiku 5.5	$0.00045	15%	$0.3000	**$0.3005**
Haiku 5.5 at 1.5%	$0.00045	1.5%	$0.0300	**$0.0305**

Look at how much the decision turns on the error rate, not the price. With a 15% error rate, the cheapest model is about eight times as costly as Opus in total. With a 1.5% rate, it's the cheapest of all. **Only your eval can tell you which row is true.**

```
record ModelOption(String id, double inputPerM, double outputPerM, double errorRate) {
    double costPerTask(long input, long output, double humanFix) {
        double model = input / 1e6 * inputPerM + output / 1e6 * outputPerM;
        return model + errorRate * humanFix;
    }
}
```

Selection criteria beyond cost

Criterion	Question
Quality	Does it pass the eval on your data?
Latency	Does it meet your TTFT and total latency targets?
Context	Does your largest realistic request fit?
Features	Does it support what you need, such as tools, batches or citations?
Data handling	Does it meet your retention and region rules? Fable 5.1, for example, requires 30-day retention
Platform	Is it available on the cloud platform you must use?

Public benchmarks are a weak guide. They measure someone else's tasks, and that's why evaluation on your own data comes first.

How Anthropic approaches it

Problem	What Anthropic provides
Tasks differ in difficulty	Multiple tiers, plus `effort` inside one model
Cost shouldn't require a new model	Prompt caching and batches
Requests might be declined	`refusal`, with server-side fallbacks you can enable
Capability data changes	A Models API exposing limits and capabilities
Teams need a method	Evaluation guidance

The production architecture

This is a recommended reference design, not a specific company's system. AI10 L2 builds the full gateway.

Baseline: hard-coded model ID. Simple, until a model is retired or a cheaper one would suffice. Every change means a redeploy.

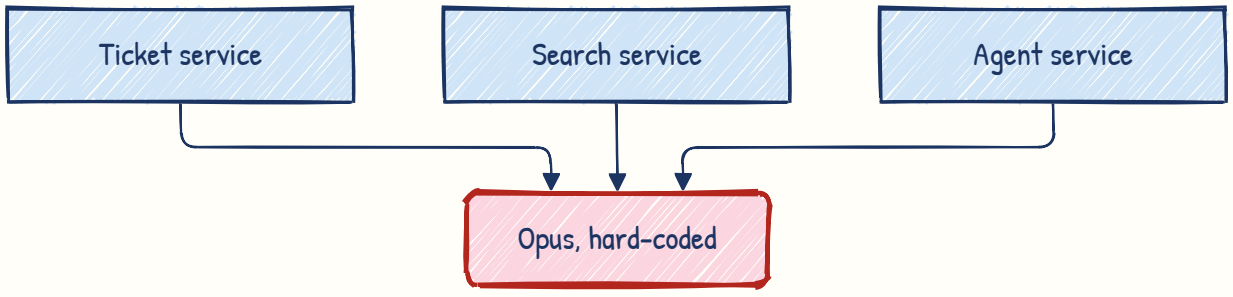


Figure 3. Baseline. The model ID is compiled into each service, so nobody can change it without a release.

Improved: central routing by task. A config maps each task to a model. New problem: **a change to the config can silently degrade quality, because routing has no connection to evaluation.**

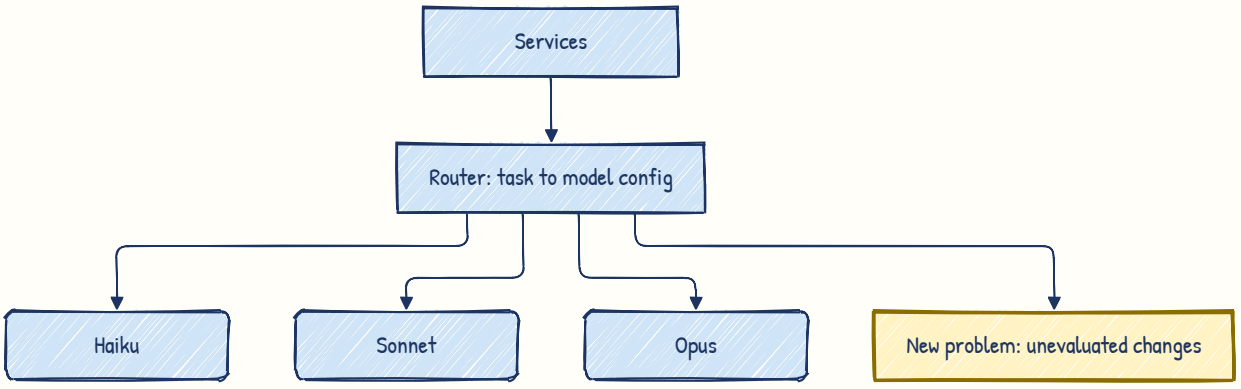


Figure 4. Improved. Models are now easy to change, and just as easy to change for the worse.

Production: routing guarded by evals.

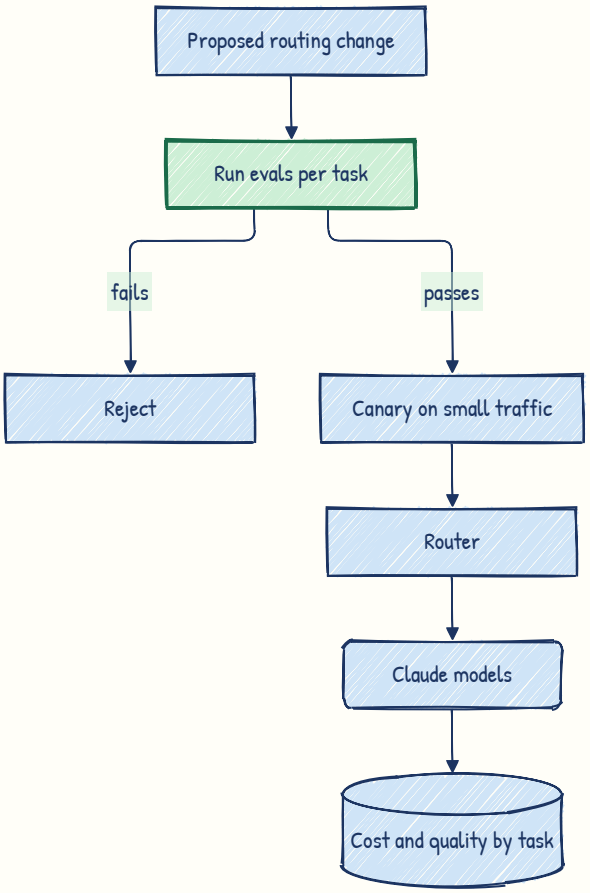


Figure 5. Production. Any routing change must pass evals first. A canary then limits the blast radius, and live cost and quality data feed the next decision.

Component	Why it exists	How it communicates
Evals per task	The only protection against silent regression	CI job with a labeled dataset (AI10 L3)
Canary	Limits a bad change	Router splits traffic
Router	One place to change model and effort	Config read by the gateway
Cost and quality data	Evidence for cost per task	Async logs by task and model

Failures

Failure	Containment
A model is deprecated	Router changes in one place, and evals run first
A provider has an outage	Fallback model, which has to be tested in advance (AI10 L1)
Cheap route's quality drifts	Sampled evals trigger an alert
A model rejects a parameter	Keep per-model request rules in the gateway

Trade-offs

Decision	Gain	Cost
One capable model everywhere	Simple, one cache namespace	May overspend on simple tasks
Lower effort on one model	Savings with no routing	Quality must be measured
Multi-model routing	Potentially large savings	Evals per route, and no shared cache
Self-hosted open model	Control	Operating everything yourself (AI11 L2)

When not to route: at low volume, the engineering time costs more than the savings. Start with one model.

Common mistakes

1. **Comparing by price per token alone.** Compare cost per completed task.
2. **Choosing from a public leaderboard.** Those aren't your tasks.
3. **Swapping model IDs without testing.** Parameters and behaviors differ.
4. **Building a cascade too soon.** Try lower effort on one model first.
5. **Hard-coding model IDs across services.** Centralize them.

What to say in an interview

Asked which model you'd pick: "It depends on the task, so I'd define success criteria and an eval set first. I'd start with a capable model to establish what's achievable, try lower effort, then step down. I'd compare cost per completed task, including the cost of errors." Then give an example of when a cheap model wins and when it loses.

Certification lens

How this lesson maps to CCAR-F

Vision, performance benchmarks, model comparisons and pricing calculations are out of scope.

The relevant habit is one the guide applies throughout: a proportionate first step, and evidence before infrastructure. Note also its sample question warning that a larger context window doesn't fix attention quality.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Claude Code for Continuous Integration · Medium. Single-pass review of a 20-file PR gives inconsistent depth and contradictory comments. A colleague suggests a higher-tier model with a larger context window. What's the better change?

A. Run per-file passes for local issues, plus a separate integration pass for cross-file data flow.

B. Accept the suggestion, since more capacity gives each file more attention.

C. Run the same single pass three times, keeping only issues found in all three.

D. Require developers to split every PR into four-file submissions.

A **A.** The root cause is attention dilution, so split the work (TS 1.6, TS 4.6). B confuses capacity with attention. C would suppress real issues that show up only intermittently. D shifts the burden to developers, without improving the system.

Q Q2 · Calculation · Medium. Requests use 10,000 input and 1,000 output tokens. Using this lesson's prices, what do 100,000 requests cost on Sonnet 5.5?

A Input: 1,000M tokens × $2 = $2,000. Output: 100M tokens × $10 = $1,000. Total **$3,000**.

Q Q3 · Match the following · Easy. Match each model family to its idea. Families: (1) GAN, (2) diffusion, (3) VAE. Ideas: (a) reverse a noising process, (b) sample from a learned compressed code, (c) competing generator and discriminator.

A 1-c, 2-a, 3-b.

Revision - Lesson 7

→ Generative models sample new examples that resemble their training data.

→ GANs compete, VAEs sample from compressed codes, and diffusion reverses noise.

→ Multimodal models turn images and text into vectors for the same attention process.

→ Listed prices vary by a factor of 100 across tiers in this lesson's example.

→ Compare cost per completed task, including the cost of errors.

→ Method: success criteria, eval set, capable model first, lower effort, then step down.

→ Try lower effort on one model before building a cascade. Caches are model-scoped.

→ Models differ in API behavior, so test them before routing.

→ Production: central routing, guarded by evals and a canary.

→ Vision, benchmarks and pricing calculations are out of scope for the exam.

Your notes

Module Revision

Generative AI and LLMs - on one page

Tokens · The unit an LLM reads and bills. Counts depend on content and on the model. Measure with count_tokens, and recount when migrating.

Attention · A soft lookup: queries compared with keys produce weights, which blend values. It makes meaning depend on context, at a quadratic comparison cost.

The transformer · Repeated blocks of attention and feed-forward, stabilized by residuals and normalization. LLMs are decoder-only, so prompt and reply share one window.

Training · Pre-training learns prediction, and alignment teaches behavior. RLHF uses human rankings. Constitutional AI uses principles and AI feedback for harmlessness.

Inference · One token at a time. Sampling explains variation. The KV cache and continuous batching make it affordable. Check stop_reason first.

Failure · LLMs produce what's plausible. Stated reasoning isn't an audit log, and raw confidence must be calibrated. Design around this with grounding, validation, code and review.

Choosing · Compare cost per completed task on your own evals. Try lower effort before a cascade.

The theme · Alignment and prompts shape probabilities. Code provides guarantees.

Self-assessment

Answer without looking back. Each has a specific answer in this module.

#	Question
1	Why did tokens rise after a model migration, with an identical prompt?
2	Walk through BPE training. What exactly is the tokenizer's output?
3	Why can JSON and IDs use more tokens than prose?
4	Is `count_tokens` exact? Is it billed? Which inputs does it reject?
5	Why reserve tokens for output?
6	What's the attention formula, read from the inside out?
7	Why divide by the square root of d_k?
8	What does a causal mask do, and why is it necessary?
9	Why must position be added to attention? Name three methods.
10	Why is attention the root of prompt injection?
11	What are the two main layers in a block, and why is each needed?
12	Why do prompt and reply share a window?
13	Do more heads add parameters?
14	Why do weights not learn from your chats?
15	Why might a pre-trained model answer a question with more questions?
16	Describe RLHF's three steps. Name three of its problems.
17	Describe Constitutional AI's two phases. What do they replace?
18	What's Claude's constitution's priority order?
19	How does preference training lead to sycophancy?
20	Compare greedy, temperature, top-k and top-p.
21	What can you control instead of temperature on current Claude models?
22	List the stop reasons, and what you do for each.
23	What does a KV cache save, and what does it cost?

#	Question
24	Why does continuous batching beat static batching?
25	What's TTFT, and when should you optimize it?
26	What did Anthropic find about the default for answering?
27	What's "lost in the middle", and what's the mitigation?
28	Why isn't a model's explanation an audit log?
29	When is a confidence score acceptable for routing?
30	What are the limits of interpretability research?
31	Compare GANs, VAEs and diffusion.
32	Why isn't price per token the same as cost per completed task?
33	What should you try before building a multi-model cascade?

Where this module is used

Idea	Where it returns
Token budgets	AI03 L5–L6, AI10 L5
Instructions and data share a sequence	AI09 L2
`stop_reason`	AI03 L1, AI06 L2, AI08 L1, TS 1.1
Batch trade-offs	AI03 L5, TS 4.5
KV cache and prefix reuse	AI03 L5, AI11 L2
Lost in the middle, attention dilution	AI03 L6, TS 5.1, TS 4.6
Fabrication under required fields	AI03 L3, TS 4.3
Unreliable reasoning and confidence	AI06 L8, AI10 L3, TS 5.2, TS 5.5
Sycophancy	AI03 L2, AI12 L5
Alignment isn't enforcement	AI06 L8, AI09, TS 1.4, TS 1.5
Cost per completed task	AI10 L2, AI10 L5

Working space